

A Technical Introduction to Word2vec

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{j \neq 0} \log p(w_{t+j} | w_t)$$

Section 1

Approach Word2vec is a group of related models that are used to produce word embeddings. These models are shallow, two-layer neural networks that are trained to reconstruct linguistic contexts of words. Word2vec takes as its input a large corpus of text and produces a mapping of the set of words to a vector space, typically of several hundred dimensions, with each unique word in the corpus being assigned a vector in the space. Word2vec can use either of two model architectures to produce these distributed representations of words: continuous bag of words (CBOW) or continuously sliding skip-gram. In both architectures, word2vec considers both individual words and a sliding context window as it iterates over the corpus. The CBOW can be viewed as a 'fill in the blank' task, where the word embedding represents the way the word influences the relative probabilities of other words in the context window. Words which are semantically similar should influence these probabilities in similar ways, because semantically similar words should be used in similar contexts. The order of context words does not influence prediction (bag of words assumption). In the continuous skip-gram architecture, the model uses the current word to predict the surrounding window of context words. The skip-gram architecture weighs nearby context words more heavily than more distant context words. According to the authors' note, CBOW is faster while skip-gram does a better job for infrequent words. After the model is trained, the learned word embeddings are positioned in the vector space such that words that share common contexts in the corpus — that is, words that are semantically and syntactically similar — are located close to one another in the space. More dissimilar words are located farther from one another in the space.

Section 2

BioVectors An extension of word vectors for n-grams in biological sequences (e.g. DNA, RNA, and proteins) for bioinformatics applications has been proposed by Asgari and Mofrad. Named bio-vectors (BioVec) to refer to biological sequences in general with protein-vectors (ProtVec) for proteins (amino-acid sequences) and gene-vectors (GeneVec) for gene sequences, this representation can be widely used in applications of machine learning in proteomics and genomics. The results suggest that BioVectors can characterize biological sequences in terms of biochemical and biophysical

interpretations of the underlying patterns. A similar variant, dna2vec, has shown that there is correlation between Needleman–Wunsch similarity score and cosine similarity of dna2vec word vectors.

Section 3

Mathematical details This section is based on expositions. A corpus is a sequence of words. Both CBOW and skip-gram are methods to learn one vector per word appearing in the corpus. Let V ("vocabulary") be the set of all words appearing in the corpus C . Our goal is to learn one vector $v_w \in \mathbb{R}^d$ for each word $w \in V$. The idea of skip-gram is that the vector of a word should be close to the vector of each of its neighbors. The idea of CBOW is that the vector-sum of a word's neighbors should be close to the vector of the word.

Section 4

Dimensionality Quality of word embedding increases with higher dimensionality. But after reaching some point, marginal gain diminishes. Typically, the dimensionality of the vectors is set to be between 100 and 1,000.

Section 5

Word2vec is a technique in natural language processing for obtaining vector representations of words. These vectors capture information about the meaning of the word based on the surrounding words. The word2vec algorithm estimates these representations by modeling text in a large corpus. Once trained, such a model can detect synonymous words or suggest additional words for a partial sentence. Word2vec was developed by Tomáš Mikolov, Kai Chen, Greg Corrado, Ilya Sutskever and Jeff Dean at Google, and published in 2013. Word2vec represents a word as a high-dimension vector of numbers which capture relationships between words. In particular, words which appear in similar contexts are mapped to vectors which are nearby as measured by cosine similarity. This indicates the level of semantic similarity between the words, so for example the vectors for walk and ran are nearby, as are those for "but" and "however", and "Berlin" and "Germany".

Section 6

The idea of CBOW is to represent each word with a vector, such that it is possible to predict a word using the sum of the vectors of its neighbors. Specifically, for each word w_i in the corpus, the one-hot encoding of the word is used as the input to the neural network. The output of the neural network is a probability distribution over the dictionary, representing a prediction of individual words in the neighborhood of w_i . The objective of training is to maximize $\sum_i \ln \Pr(w_i | \sum_{j \in N} w_j)$

$w_{i+j} \mid j \in N$ where N is a set of (non-zero) indices representing the relative locations of nearby words considered to be in w_i 's neighborhood. For example, if we want each word in the corpus to be predicted by every other word in a small span of 4 words. The set of relative indexes of neighbor words will be: $N = \{-2, -1, +1, +2\}$, and the objective is to maximize $\sum_i \ln \Pr(w_i \mid w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2})$. In standard bag-of-words, a word's context is represented by a word-count (aka a word histogram) of its neighboring words. For example, the "sat" in "the cat sat on the mat" is represented as {"the": 2, "cat": 1, "on": 1}. Note that the last word "mat" is not used to represent "sat", because it is outside the neighborhood $N = \{-2, -1, +1, +2\}$. In continuous bag-of-words, the histogram is multiplied by a matrix V to obtain a continuous representation of the word's context. The matrix V is also called a dictionary. Its columns are the word vectors. It has D columns, where D is the size of the dictionary. Let d be the length of each word vector. We have $V \in \mathbb{R}^{d \times D}$. For example, multiplying the word histogram {"the": 2, "cat": 1, "on": 1} with V , we obtain $2v_{\text{the}} + v_{\text{cat}} + v_{\text{on}}$. This is then multiplied with another matrix V' of shape $\mathbb{R}^{D \times d}$. Each row of it is a word vector v' . This results in a vector of length D , one entry per dictionary entry. Then, apply the softmax to obtain a probability distribution over the dictionary. This system can be visualized as a neural network, similar in spirit to an autoencoder, of architecture linear-linear-softmax, as depicted in the diagram. The system is trained by gradient descent to minimize the cross-entropy loss. In full formula, the cross-entropy loss is: $-\sum_i \ln \left(\frac{e^{v_{w_i} \cdot (\sum_{j \in N} v_{w_{j+i}})}}{\sum_{w'} e^{v_{w_i} \cdot (\sum_{j \in N} v_{w_{j+i}})}} \right)$ where the outer summation $\sum_i$ is over the words in a corpus, the quantity $\sum_{j \in N} v_{w_{j+i}}$ is the sum of a word's neighbors' vectors, etc. Once such a system is trained, we have two trained matrices V, V' . Either the column vectors of V or the row vectors of V' can serve as the dictionary. For example, the word "sat" can be represented as either the "sat"-th column of V or the "sat"-th row of V' . It is also possible to simply define $V' = V^T$, in which case there would no longer be a choice.